SOF Action Semantics

Contextual Interpretation and Bounded Candidate Dispositions

WuJun Chen
Independent Researcher
2026

This paper is Paper XIV of the RIME program. It consumes sparse typed audit signatures from Paper XIII and owns context- and policy-relative interpretation and bounded candidate dispositions. Selection, authorization, outcome, and effect remain downstream.

Abstract

Problem. An aligned SOF audit may contain nonzero coordinates, licensed transformation differences, or unresolved coordinates. None of these carries action meaning by itself. A direct map from mismatch counts to repair, rollback, or deploy conflates semantic interpretation with policy.

Approach. This paper introduces an explicit typed action object

$$I_{\text{interp}} = \text{Interpret}(\Delta_{\text{audit}}, K_{\text{ctx}}, \Pi_{\text{policy}}), \qquad A_{\text{cand}} = \text{Generate}(I_{\text{interp}}, K_{\text{ctx}}, \Pi_{\text{policy}}).$$

The context K_{ctx} records actor, scope, objective, constraints, time, authority, and uncertainty conditions. The profile Π_{policy} is a versioned Policy Profile with applicability, rules, typed exceptions, and an explicit precedence graph. I_{interp} contains policy-relative Interpretation Records, and A_{cand} contains bounded Candidate Actions. Missing context or an inapplicable policy produces `NoDisposition`, an empty Candidate Action Set, and no Interpretation Record or affirmative candidate.

Results. Four protocol propositions are stated: No Action Without Context and Policy, Interpretation Relativity, Action Non-Fabrication, and Audit Preservation. The controlled validation preserves the Paper XIII projection, rejects unresolved or undeclared coordinates as action support, binds every candidate to audit coordinates, carrier, context, policy rules, preconditions, declared risk considerations, reversibility, evidence, and authorization state, and keeps selection downstream. The controlled workbench validates 29 v2 objects: 28 migrated Paper XIII records remain unresolved, while one native GridWorld F4 audit yields only policy-relative review candidates.

Implications. This paper does not provide a universal repair theorem, a decision engine, or an action-effect certificate. It defines a reusable SOF action object in which difference is interpreted only under explicit context and policy. Causal effect estimation, feasibility, cost, authorization, and final policy choice remain separate downstream problems.

1 Introduction

Paper XIII associates an admissible SOF comparison object

$$\mathfrak{C}_{\text{cmp}} = (\mathcal{R}^{\star}, \widehat{\mathcal{R}}, \Phi; \Theta)$$

with a sparse typed audit signature

$$\text{Compare}(\mathfrak{C}_{\text{cmp}}) = \Delta_{\text{audit}}.$$

The comparison signature answers how two aligned reports differ. It does not answer whether that difference is a defect, whether intervention is justified, or which operational choice should be made. These questions require information that is absent from the signature coordinates themselves.

The decisive counterexample is a conforming transformation. Relocating an obstacle, retiming a traffic controller, or applying a declared compiler pass can produce nonzero support, bridge, depth, response, or wall-record differences while satisfying every transformation invariant. The same numerical pattern that indicates failure under one comparison role may indicate licensed change under another. Therefore

$$\text{difference} \not\Rightarrow \text{defect} \not\Rightarrow \text{severity} \not\Rightarrow \text{action}.$$

This paper makes four contributions.

1. It defines the typed objects K_{ctx}, Π_{policy}, I_{interp}, and A_{cand}.
2. It states No Action Without Context and Policy and Interpretation Relativity.
3. It states Action Non-Fabrication and Audit Preservation as executable invariants.
4. It provides a schema, validator, hostile fixtures, and a controlled workbench whose unresolved inputs remain unresolved.

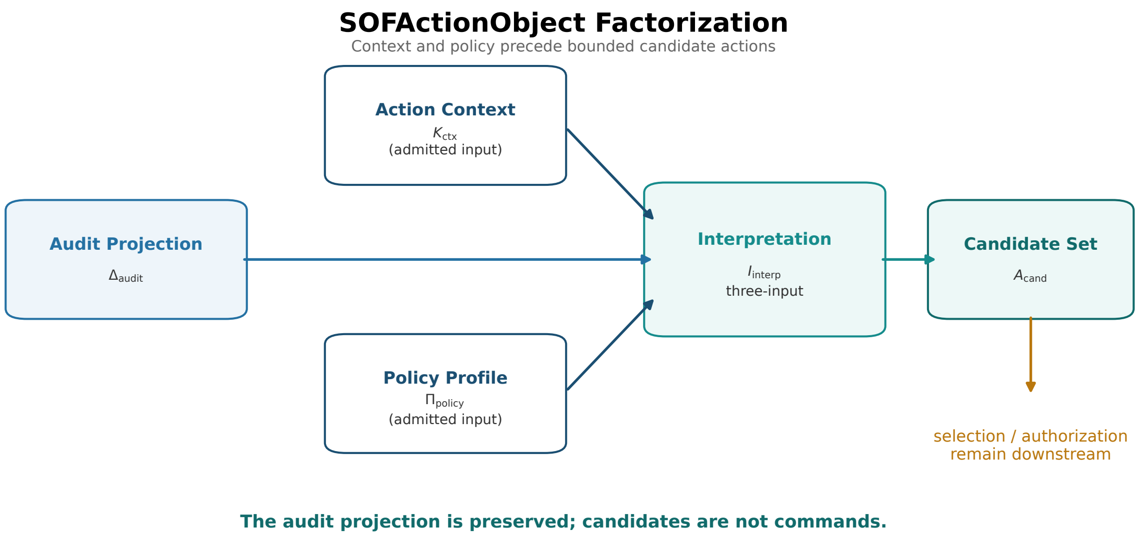

Figure 1. SOFActionObject factorization. The Paper XIII audit projection is retained, then interpreted under independently admitted ActionContext and PolicyProfile before bounded CandidateActions are generated. Selection is a separate downstream artifact.

1.1 Scope

The object studied here is semantic, not causal or optimal. An intended diagnostic consequence records what a candidate is designed to change in a subsequent audit; it is not an identified causal effect or a prediction. A precondition records when a candidate would be meaningful; it is not a feasibility proof. A Candidate Action Set records supported candidate families; it is not an executed or selected plan.

Notation Table

Symbol	Meaning
Δ_{audit}	immutable sparse typed audit projection consumed from Paper XIII
K_{ctx}	`ActionContext`, independently admitted and never derived from the audit
Π_{policy}	`PolicyProfile`, the sole normative rule input in the v2 contract
I_{interp}	`InterpretationRecord` output relative to the admitted context and policy
A_{cand}	bounded `CandidateActionSet` output
`DispositionResult`	result class closing the interpretation and candidate sets
selected plan, authorization, outcome, and effect	downstream reserved contracts, not fields or conclusions of `.sofaction`

The table names the objects owned here or consumed as typed inputs; selection and external authority approval remain downstream contracts.

2 Related Work and Novelty Boundary

Program interfaces. Paper X supplies capability-aware compilation and evidence gating, Paper XI supplies typed wall records, Paper XII supplies the single-report protocol, and Paper XIII supplies explicit aligned comparison objects [2, 5, 3, 4]. This paper consumes the sparse audit projection from Paper XIII; it does not revise report admission, alignment, wall ownership, or compiler soundness.

2.1 Causal Intervention

Structural causal models distinguish observational association from interventional claims and provide identification criteria for causal effects [8]. This paper does not identify effects of its candidate actions. Its intended-diagnostic-consequence fields are targets conditional on declared preconditions. A causal model would be an additional domain-specific input required before counterfactual or intervention-effect claims could be made.

2.2 Model-Based Diagnosis

Reiter's diagnosis theory computes diagnoses from a system description and observations that conflict with intended behavior [9]. This paper shares the insistence that discrepancy alone is insufficient, but it does not infer minimal faulty component sets. It interprets typed SOF audit coordinates relative to an explicit comparison context and emits candidate families rather than diagnoses.

2.3 Action Languages and Planning

STRIPS represents actions through state-transforming operators, while later action-language work formalizes descriptions of action effects [6, 7]. This paper borrows the discipline of explicit targets, preconditions, effects, and limitations. It does not solve a planning problem, define executable transition semantics, or claim closure and composition laws for an Action Algebra.

2.4 Decision and Policy Separation

Classical decision theory and dynamic programming formalize preferences, constraints, consequences, and sequential policy choice [10, 1]. Those theories clarify why policy is an explicit normative input to interpretation, while selection and authorization remain downstream. A policy rule may support a candidate without making that candidate objectively correct.

Novelty boundary. This paper contributes the context-indexed interpretation interface, the closed Policy Predicate Language, uncertainty propagation, and bounded candidate dispositions. It does not provide causal effect estimation, feasibility, authorization, optimal selection, post-action observation, or an Action Effect Certificate.

3 Semantic Factorization

3.1 Audit Signatures

Write the Paper XIII output as

$$\Delta_{\text{audit}} = (\Delta_1, \dots, \Delta_m).$$

The signature contains the typed coordinates requested by the Paper XIII Audit Profile. Legacy `action_response_failure` appears only in archived v1 inputs; it is not an active intervention field in this contract.

3.2 Action Context and Policy Profile

Definition 3.1 (Action Context)**.**

The context object is

$$K_{\text{ctx}} = (\text{actor}, \text{scope}, \text{objective}, \text{constraints}, \text{time}, \text{authority}, \text{uncertainty}).$$

It also retains the comparison role, mismatch direction, contract status, and a non-certifying evaluator-qualification note. The validator checks the audit identifier, reference-to-target direction, provenance-relative role, actor, and authority scope. These fields describe applicability and responsibility; they do not alter the audit or create a qualification certificate.

Definition 3.2 (Policy Profile)**.**

A versioned policy profile is

$$\begin{aligned}\Pi_{\text{policy}} = (&\text{id}, \text{contract version}, \text{revision}, \text{applicability},\\ &\text{rules}, \text{exceptions}, \text{precedence edges}).\end{aligned}$$

A policy profile supplies normative basis, uncertainty handling, candidate dispositions, and rule precedence. It does not function as a ground-truth oracle, nor does it select or authorize an action merely by matching a rule.

The admission contract treats both objects independently. Admission separates contract validation, applicability, and completeness. A missing field or unsupported contract is rejected as incomplete, whereas a valid but inapplicable policy is recorded separately as not applicable. Either state stops interpretation. The absence of context or policy is therefore not evidence of failure.

3.3 Policy Predicate Language

SOFAction v2 freezes Policy Predicate Language v1.0 as a closed, recursively typed expression language. Boolean nodes are `all`, `any`, and `not`. Leaf nodes are limited to coordinate existence, state, carrier, and relation; comparison role and contract status; authority and uncertainty status; declared context-constraint status; transformation-contract presence; and policy-basis presence. The coordinate identifier `*` denotes only the current coordinate in the per-coordinate interpreter. It is not a dynamic field path.

Predicates cannot execute code, query a network, inspect undeclared fields, or infer a condition from free text. `UNRESOLVED`, `NOT_DECLARED`, `INCOMPARABLE`, and `NOT_APPLICABLE` remain explicit states; none is coerced to false or zero. A typed exception has its own predicate and an explicit set of rules that it suppresses. `precedence_edges` is the sole precedence source. If several active rules match and no one rule precedes all other matches, the interpreter emits `policy_conflict` rather than selecting by declaration order. Deterministic replay re-executes the predicate tree and recomputes both the selected rule and the assessment kind from the frozen audit, context, and policy. Uncertainty handling is itself a versioned machine object, not a free-text instruction. Policy Predicate Language v1.0 uses $\mathbb{T}_3 = \{\mathrm{TRUE}, \mathrm{FALSE}, \mathrm{UNRESOLVED}\}$ with $\neg U = U$, $T \wedge U = U$, $F \wedge U = F$, $T \vee U = T$, and $F \vee U = U$. The `uncertainty_policy` object fixes unavailable-coordinate, `NOT_DECLARED`, incomparable, conflict, and no-applicable-rule behavior. Thus deterministic replay is claimed only for the same normalized $(\Delta_{\mathrm{audit}}, K_{\mathrm{ctx}}, \Pi_{\mathrm{policy}})$ and the same interpreter/profile version closure.

3.4 Interpretation Records

Definition 3.3 (Interpretation Record)**.**

An interpretation record is

$$\begin{aligned} I_{\mathrm{interp}} = (&\text{audit coordinate refs},\ \text{context refs},\\ &\text{policy rule refs},\ \text{assessment},\\ &\text{uncertainty},\ \text{negative boundary}). \end{aligned}$$

It may describe licensed change, a defect candidate, no action indicated, or an inconclusive state. It never rewrites the source coordinate or promotes `UNRESOLVED`, `NOT_DECLARED`, or a declared baseline into a defect.

Severity, confidence, and assessment are policy- and context-relative; they are not intrinsic magnitudes of Δ_i. Every record retains the source coordinate state, the context identifier, the applicable policy rules, and its negative boundary.

3.5 Candidate Actions

Definition 3.4 (Candidate Action)**.**

A candidate action is

$$\begin{aligned} A_{\mathrm{cand}} = (&\text{id},\ \text{preconditions},\\ &\text{intended diagnostic consequence},\\ &\text{declared risk considerations},\ \text{reversibility},\\ &\text{evidence refs},\ \text{authorization state}). \end{aligned}$$

Each candidate also carries its disposition, target coordinate, context reference, policy-rule references, and negative boundary. The disposition type is the explicit sum

$$\begin{aligned}\text{CandidateActionKind} = \text{Investigate} &\sqcup \text{RequestEvidence} \sqcup \text{Mitigate} \\ &\sqcup \text{Rollback} \sqcup \text{Escalate}.\end{aligned}$$

The result layer is separate:

$$\begin{aligned}\text{DispositionResult} = \text{NoDisposition} &\sqcup \text{UnresolvedDisposition} \\ &\sqcup \text{NoActionDisposition} \sqcup \text{CandidateActionSet}.\end{aligned}$$

An empty Candidate Action Set is not itself `NoAction`: `NoDisposition` means that no legal disposition was formed, `UnresolvedDisposition` means that admitted inputs remain insufficient, and `NoActionDisposition` is an explicit policy-supported disposition.

These are reusable types, not a universal operational vocabulary. A candidate is not an execution command, a recommendation, a feasibility proof, a causal effect, or an authorization receipt. The v2 contract permits candidates to record `not_requested`, `required`, `pending`, or `denied`; they cannot declare themselves authorized. Authorization is reserved for a future external `.sofauth` contract.

3.6 The Executable Factorization

The canonical action object is

$$\boxed{\text{SOFActionObject} = (\Delta_{\text{audit}}, K_{\text{ctx}}, \Pi_{\text{policy}}, I_{\text{interp}}, A_{\text{cand}})}$$

with the admitted-input construction

$$I_{\text{interp}} = \text{Interpret}(\Delta_{\text{audit}}, K_{\text{ctx}}, \Pi_{\text{policy}}), \qquad A_{\text{cand}} = \text{Generate}(I_{\text{interp}}, K_{\text{ctx}}, \Pi_{\text{policy}}).$$

The ActionContext and PolicyProfile are independent admitted inputs; neither is derived from the audit signature. The action generator accepts Interpretation Records and an admitted Policy Profile; it rejects a raw SOFAUDIT payload. Every candidate stores the audit coordinate, carrier, context, policy rule, preconditions, declared risk considerations, reversibility, evidence references, and authorization state that support it.

The v2 contract claims phase separation, not a fully split semantic rulebook and action-generation profile. The same admitted `PolicyProfile` is therefore carried into both phases: `Interpret` evaluates its predicate and precedence semantics, while `Generate` uses the resulting rule references and allowed disposition closure to bind candidates. `Generate` does not re-interpret the audit or select an action. A future split into a semantic `Rulebook` and a candidate-generation profile is a Research Program item, not a property of the v2 artifact contract.

4 Core Propositions

Proposition 4.1 (No Action Without Context and Policy)**.**
There is no admitted mapping $\Delta_{\text{audit}} \mapsto A_{\text{cand}}$ under this contract without an admitted K_{ctx} and an applicable Π_{policy}. The permitted construction is

$$I_{\text{interp}} = \text{Interpret}(\Delta_{\text{audit}}, K_{\text{ctx}}, \Pi_{\text{policy}}), \qquad A_{\text{cand}} = \text{Generate}(I_{\text{interp}}, K_{\text{ctx}}, \Pi_{\text{policy}}).$$

Proof. The Action Object contract requires context and policy admission before an Interpretation Record can be emitted. The candidate validator requires each candidate to reference both objects. A missing or inapplicable input therefore yields `NoDisposition` and an empty Candidate Action Set rather than a default action.

Proposition 4.2 (Interpretation Relativity)**.**
For one retained audit signature, two admitted policy/context pairs may produce different but valid interpretation records:

$$I_{K_1,\Pi_1}(\Delta) \neq I_{K_2,\Pi_2}(\Delta).$$

This is a change in normative input, not an instability of the interpreter.

Proof. The rule precedence and applicability fields are part of the input to interpretation. A conforming transformation may be assessed as `licensed_change`, while the same active coordinate under a failure-control context may be assessed as a `defect_candidate`. Both records preserve the same audit coordinate and cite their own context and policy rule.

Proposition 4.3 (Action Non-Fabrication)**.**
Every generated candidate action references at least one retained audit coordinate, one admitted context, and one applicable policy rule. It also records preconditions, intended diagnostic consequence, declared risk considerations, reversibility, evidence references, authorization state, and a negative boundary. No candidate is generated from a missing, `UNRESOLVED`, or `NOT_DECLARED` coordinate alone.

Proposition 4.4 (Audit Preservation)**.**
For every valid $S = (\Delta_{\text{audit}}, K_{\text{ctx}}, \Pi_{\text{policy}}, I_{\text{interp}}, A_{\text{cand}})$*,*

$$\text{AuditProjection}(S) = \Delta_{\text{audit}}.$$

Interpretation and candidate generation may add references and bounded semantic fields, but may not rewrite comparison state, reference authority, evidence level, or source provenance.

Proof. The v2 artifact embeds the sparse Paper XIII coordinate projection, binds the source audit and its Paper XIII validation receipt by SHA-256, and compares decoded projections for structural equality. Canonical JSON encoding then gives a distinct artifact-level byte comparison; these two equality checks are not conflated. Candidate support is checked against the same coordinate IDs, carriers, and states.

The controlled transformation fixtures instantiate Interpretation Relativity, while the native GridWorld F4 chain supplies a factual v2 input. The 28 migrated Paper XIII records remain unresolved and therefore do not supply affirmative candidates. The resulting boundary is:

$$\text{difference} \not\Rightarrow \text{defect} \not\Rightarrow \text{severity} \not\Rightarrow \text{action}.$$

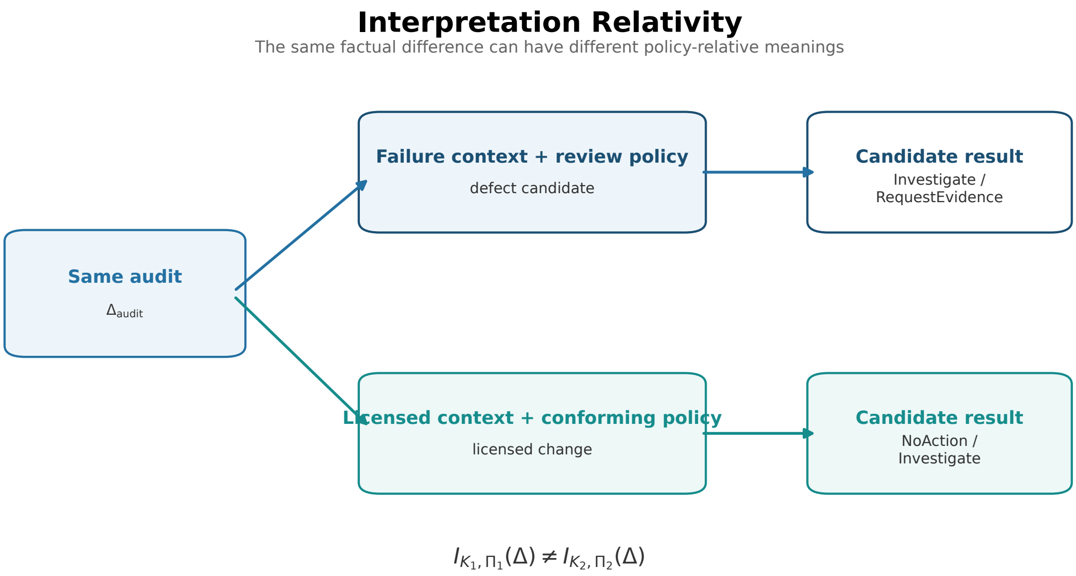

Figure 2. Context and policy relativity. One raw signature can be interpreted as a reference violation under one admitted context and as licensed change under a conforming transformation contract. The audit projection remains unchanged.

5 Candidate Action Sets

5.1 Structured Candidates

The candidate set is generated only from admitted Interpretation Records:

$$A_{\text{cand}} = \bigcup_{i \text{ admitted}} \text{Generate}_{K_{\text{ctx}},\Pi_{\text{policy}}}(I_{\text{interp},i}).$$

It may be empty. The controlled workbench uses only `Investigate` and `RequestEvidence` for the native factual GridWorld F4 mismatch. Explicit `NoActionDisposition`, `UnresolvedDisposition`, and `NoDisposition` remain distinct from the Candidate Action Set. No domain is required to implement every CandidateActionKind.

> **Protocol Invariant (Action Non-Fabrication).** A candidate produced by the v2 generator cites an audit coordinate, an admitted ActionContext, and an applicable PolicyProfile rule. It does not cite an unresolved coordinate as affirmative support, and it does not contain an execution command or an unverified causal effect.

The invariant is a contract property for the declared policy profile. It does not show that a candidate is feasible, causally effective, safe, authorized, or optimal. Those require external domain models and downstream governance.

5.2 Typed Channel Semantics

Word and Lie bridge channels remain distinct throughout the factorization. A word discrepancy supports composition-path candidates; a Lie discrepancy supports commutator-channel candidates. The generator does not collapse them into a generic bridge score.

The GridWorld F4 control provides a strict implementation witness: a Lie-channel difference generates a Lie-carrier candidate without a word-bridge insertion candidate. This is compatible

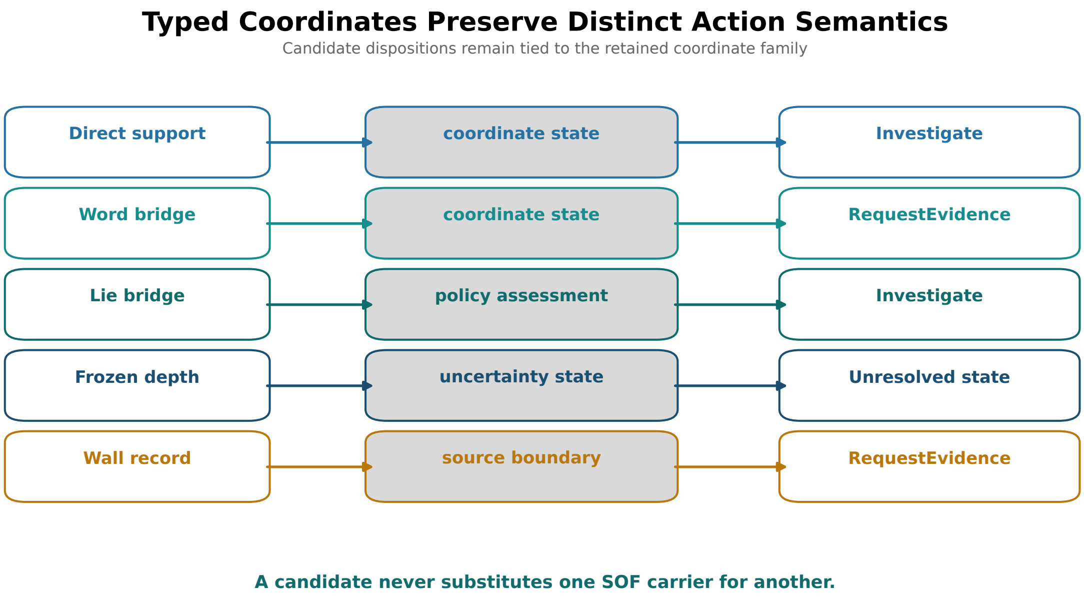

Figure 3. Typed coordinate semantics. Direct support, word bridges, Lie bridges, frozen depth, and wall records retain distinct meanings and candidate families through semantic interpretation and action generation.

with the static word/Lie carrier separation owned by Paper VIII and preserved by Paper XIII's aligned comparison contract; it is not a universal claim that every domain realizes both channels.

5.3 Empty and Inconclusive Cases

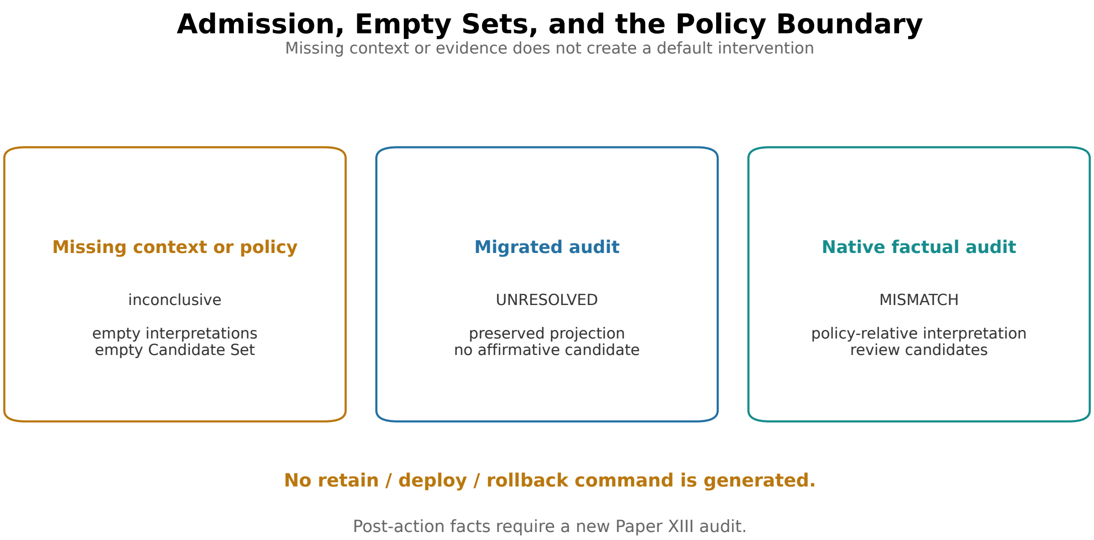

Figure 4. Semantic admission and empty-set boundary. Missing context and admitted zero signatures both produce empty Action Sets. No default retain, deploy, or rollback decision is generated.

Three result states are distinguished.

1. `NoDisposition` means that no legal disposition was formed, including when context or policy admission stops the chain or no coordinates are retained.
2. `UnresolvedDisposition` means that context and policy were admitted but unresolved, not-declared, incomparable, or not-applicable coordinates block affirmative disposition.
3. `NoActionDisposition` means that the applicable policy explicitly supports no action. It is not an empty set accidentally interpreted as safety.

An admitted active signature may instead produce a nonempty Candidate Action Set. Empty output has no universal safety meaning; its result kind records why no candidate action was formed.

6 The SOF Action Contract

The canonical machine-readable contract is the versioned SOFAction v2 schema (Artifact A1). Its principal fields are:

Field	Role
Source audit / projection	immutable Paper XIII source and validation-receipt references plus preserved coordinate map
Action context	explicit actor, scope, objective, constraints, time, authority, and uncertainty
Policy profile	contract version, policy revision, source-addressed normative basis, typed predicates and exceptions, and precedence edges
Interpretations	coordinate, context, and policy references with assessment and negative boundary
Candidate set	zero or more bounded CandidateAction records
Disposition result	explicit `NoDisposition`, `UnresolvedDisposition`, `NoActionDisposition`, or Candidate Action Set state
Record class / basis	v2 policy-conformance or decision-trace class with source-addressed protocol basis
Failure modes	non-implication, applicability, and epistemic boundaries

The schema is closed: unknown predicate, context, policy, interpretation, and candidate fields are rejected. Normative evidence cannot be a bare string. Semantic conformance additionally requires source-receipt and digest closure, exact Audit Projection preservation, context and policy admission, deterministic predicate and precedence replay, interpretation and candidate closure, authority-scope closure, and disposition consistency. Appendix A indexes the corresponding executable controls.

The `.sofaction` v2 contract has only two record classes:

Class	What it can establish	What it cannot establish
Policy Conformance Certificate	typed policy predicates were applied under the declared contract and revision	policy validity or action correctness
Decision Trace Certificate	audit, context, policy, interpretation, and candidate links are complete	feasibility, safety, causal effect, authorization, or optimality

Four related concepts remain reserved for separate contracts rather than labels inside `.sofaction`: `.sofplan` for a selected plan, `.sofauth` for an authorization receipt, `.sofoutcome` for a post-action observation, and `.sofeffect` for an independently validated intervention effect. In particular, an Outcome Observation cannot be relabelled as an Action Effect Certificate, and an authorization receipt is not a scientific evidence level.

Canonical `.sofaction` artifacts stop at the Candidate Action Set. An optional selector consumes that set and emits a separate downstream plan artifact; it cannot modify the audit projection, interpretations, or candidate set:

$$\pi : \mathsf{ActionSet} \times \mathsf{PolicyContext} \to \mathsf{SelectedActionPlan}.$$

Policy may encode cost, safety requirements, authorization, deployment stage, or service objectives, but matching a policy rule does not make a candidate objectively correct. These quantities are not coordinates of the Paper XIII audit projection.

7 Claim Spine

Definitions and negative ownership boundaries are not additional reader-facing evidence levels. Certificate classes identify what a finite validation establishes; they do not promote policy correctness or candidate effectiveness.

Object or conclusion	Formal role and claim target	Reader-facing status
`SOFActionObject`, `ActionContext`, `PolicyProfile`, `InterpretationRecord`, and `DispositionResult`	owned type definitions; representation interface	not an independent evidence claim
No Action Without Context and Policy	domain-of-definition proposition; representation interface	Theorem
Interpretation Relativity	context/policy-relative proposition; bounded fixtures provide a controlled witness	Theorem
Action Non-Fabrication and Audit Preservation	exact representation-interface propositions under the v2 contract	Theorem
Versioned Predicate Replay	finite protocol-conformance replay under one normalized input and version closure; not policy correctness	Computational Certificate
29-object workbench	finite schema and semantic validation; Policy Conformance / Decision Trace Certificate	Computational Certificate
Native GridWorld F4 candidates	policy-relative bounded outputs; no feasibility, authorization, selection, or effect claim	Computational Observation
action effectiveness and optimal selection	open downstream problems reserved for external contracts	Research Program

Generation of an `Investigate` candidate therefore does not establish that investigation is the correct action. A protocol trace proves closure of the declared inputs and links, not scientific adequacy, causal effect, feasibility, safety, or authorization.

8 Controlled Validation

The controlled v2 workbench consumes 28 migrated Paper XIII SOFAUDIT records and one native GridWorld F4 factual audit. All 29 objects retain an exact audit projection, an explicit ActionContext, and an applicable PolicyProfile. The migrated records contain only unresolved or not-declared coordinate states and therefore produce no affirmative candidates. The native F4 record produces only `Investigate` and `RequestEvidence` candidates under the declared review policy.

Representative controls are summarized below.

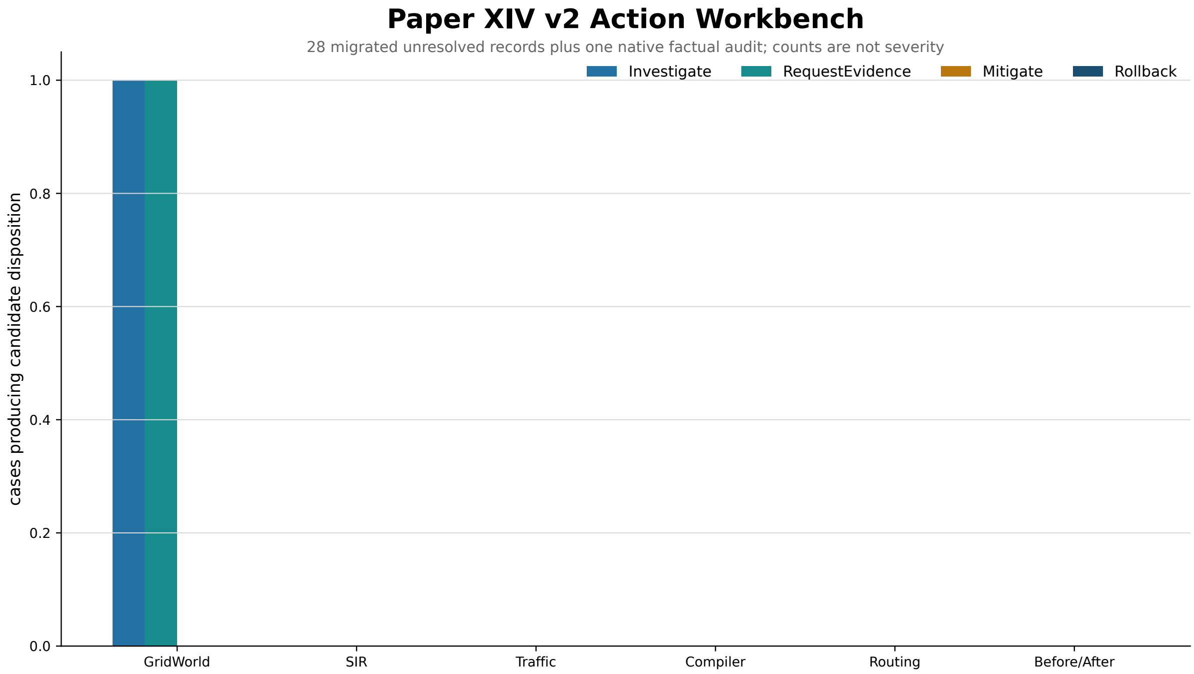

Figure 5. Controlled semantic workbench. The workbench distinguishes migrated unresolved inputs from the native factual GridWorld F4 trace. Candidate counts describe policy-relative coverage, not defect severity or system quality.

Case	Active semantic pattern	Derived boundary
28 migrated records	unresolved or not-declared coordinates	inconclusive interpretation; no candidate action
Native GridWorld F4	certified simple-commutator mismatch with aligned support channels	investigation and evidence request only
Hostile policy fixtures	malformed rules, projection rewrite, unresolved support, carrier or authority substitution	validator rejection

The counts are not quality or severity scores. They report the result of a declared policy over the declared evidence states. Candidate feasibility, authorization, causal effect, and safety are not tested by this workbench.

9 Claim Boundary

This paper establishes the typed Action Object and four protocol propositions. It does not claim:

- a universal repair theorem,
- causal identification of candidate effects,
- domain-independent feasibility,
- a universal or complete action vocabulary,
- cost, risk, or authorization calibration,
- optimal policy selection,
- that a policy rule match makes a candidate objectively correct,

- an Action Effect Certificate under the `.sofaction` contract,
- that contract conformance proves system correctness,
- or composition, conflict, identity, and closure laws for an Action Algebra.

The workbench is controlled methodological evidence. Production use requires domain intervention models, authorization, causal or empirical validation, and post-intervention measurement.

10 Relation to Papers XII and XIII

The protocol boundary is now:

Paper	Mathematical object	Artifact	Question
XII	single-system diagnostic report	.sofreport	What was measured?
XIII	aligned comparison object and factual signature	.sofaudit	How do two reports differ?
XIV	`SOFActionObject = (Delta_audit, K_ctx, Pi_policy, I_interp, A_cand)`	.sofaction	Under which context and policy can a difference be interpreted and which bounded candidates are supported?
Downstream selector	objective-relative selector	reserved .sofplan	Which candidate should be chosen?

This separation allows the same factual audit record to be interpreted under different admissible contexts or policies without rewriting Paper XIII evidence. It does not turn the selected reference into ground truth.

11 Outlook

Several extensions remain open.

1. Add domain-specific policy profiles and test their authority and exception semantics under the frozen predicate language.
2. Define action compatibility, conflict, composition, identity, and equivalence before introducing Action Algebra terminology.
3. Add domain causal models that can turn intended diagnostic consequences into testable intervention predictions.
4. Study minimal candidate sets and residual-signature prediction without conflating either problem with policy optimality.
5. Develop longitudinal records in which post-intervention SOF Reports close the measurement loop without rewriting the original audit.

12 Conclusion

This paper fixes the semantic object between comparison and any downstream disposition:

$$\mathrm{SOFActionObject} = (\Delta_{\mathrm{audit}}, K_{\mathrm{ctx}}, \Pi_{\mathrm{policy}}, I_{\mathrm{interp}}, A_{\mathrm{cand}}).$$

The central boundary is

$$\text{difference} \not\Rightarrow \text{defect} \not\Rightarrow \text{severity} \not\Rightarrow \text{action}.$$

An audit is preserved, context and policy are explicit, interpretation is relative to those inputs, and candidates remain bounded records rather than execution commands. This is the stable interface required before SOF diagnostics can participate in intervention workflows.

Appendix A Computational Artifacts

A.1 Contract and Reference Implementation

The following source-addressed artifacts implement or validate the v2 `.sofaction` contract. The schema is the normative machine-readable shape; the engine, workbench, and validator are reference implementations and evidence producers, not semantic authorities.

Artifact	Role	Source-addressed path
A1	SOFActionObject schema and record-class contract	`schemas/sofaction/v2.0.schema.json`
A2	ActionContext/PolicyProfile admission, interpretation, and candidate engine	`experiments/paper14/action_engine.py`
A3	controlled 29-object workbench	`experiments/paper14/action_workbench.py`
A4	semantic validator and receipt producer	`experiments/paper14/validate_sofaction.py`
A5	optional downstream selector; excluded from canonical `.sofaction` evidence	`experiments/paper14/policy_selector.py`
A6	generated v2 action artifacts and validation receipts	`experiments/paper14/results/`
A7	focused hostile tests and checked-artifact/receipt closure	`tests/test_sof_action.py`; `tests/test_sofaction_v2.py`
A8	public reference runtime, wheel-install external-adopter workflow, and Level 3 conformance surface	`sof-runtime` v0.1.0

Validation checks include schema closure, source digest closure, exact Audit Projection preservation, Paper XIII validation-receipt binding, context and policy admission, policy applicability, independent predicate replay, precedence graph acyclicity, typed exception coverage, one-to-one coordinate interpretation coverage, hostile unresolved support rejection, candidate-to- interpretation and carrier references, authority actor/scope closure, and canonical omission of downstream selection.

A.2 Experimental Reproduction

The workbench reads the 28 migrated Paper XIII audits plus the native GridWorld F4 audit and regenerates 29 v2 Action Objects. The runnable reproduction entry points are indexed in `experiments/paper14/README.md`. Figures summarize the formal factorization and controlled outputs; they are not independent evidence.

A.3 Boundary Controls

The automated controls include:

- rejection of raw audit payloads by the candidate generator;
- policy/context relativity witnesses on one native audit;

- exact Audit Projection preservation;
- rejection of unknown policy rules and inapplicable profiles;
- rejection of arbitrary predicate nodes, bare evidence strings, and forged interpretation assessments;
- rejection of dispositions supported by `UNRESOLVED` coordinates;
- rejection of precedence cycles, conflicting rules, and uncovered exceptions;
- rejection of wrong carriers, out-of-scope actors, attempts to self-declare `authorized`, and selector IDs outside a Candidate Set;
- explicit inconclusive output for missing context or policy;
- and omission of policy selection and post-action observations from canonical workbench artifacts.

The tagged runtime release is a public reference implementation, not a normative definition source; its vendored contracts are independently bound to the upstream release-content commit or isolated as explicit candidates. These controls establish implementation fidelity to the formal interface. They do not establish domain intervention effectiveness. A1–A8 establish only their declared schema, protocol-conformance, replay, and bounded-output targets; they do not establish policy correctness, feasibility, authorization, selection, post-action outcome, or causal effect. Full generated paths and digests are indexed in `experiments/paper14/README.md`; all listed artifacts are available in the RIME repository.

References

[1] Richard Bellman. *Dynamic Programming*. Princeton University Press, Princeton, New Jersey, 1957.

[2] WuJun Chen. Capability-aware compilation for sectorized observable frameworks: Typed admission and cross-species registry evidence. *Preprint*, 2026. Paper X of the RIME program, version 2.0. DOI: https://doi.org/10.5281/zenodo.21768257.

[3] WuJun Chen. SOF diagnostic protocol: Realization-relative reports for strict SOF realizations and diagnostic analogues. *Preprint*, 2026. Paper XII of the RIME program, version 2.0. DOI: https://doi.org/10.5281/zenodo.21870368.

[4] WuJun Chen. Typed alignment and audit signatures: Realization-relative comparison for SOF reports. *Preprint*, 2026. Paper XIII of the RIME program, version 2.0. DOI: https://doi.org/10.5281/zenodo.21870778.

[5] WuJun Chen. Typed wall morphology for sectorized observable frameworks: Sparse wall records, coordinate profiles, multi-label taxonomy, and local-model eligibility. *Preprint*, 2026. Paper XI of the RIME program, version 2.0. DOI: https://doi.org/10.5281/zenodo.21801722.

[6] Richard E. Fikes and Nils J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.

[7] Michael Gelfond and Vladimir Lifschitz. Action languages. *Electronic Transactions on Artificial Intelligence*, 3(16):195–210, 1998.

[8] Judea Pearl. Causal diagrams for empirical research. *Biometrika*, 82(4):669–688, 1995.

[9] Raymond Reiter. A theory of diagnosis from first principles. *Artificial Intelligence*, 32(1):57–95, 1987.

[10] Leonard J. Savage. *The Foundations of Statistics*. Wiley, New York, 1954.